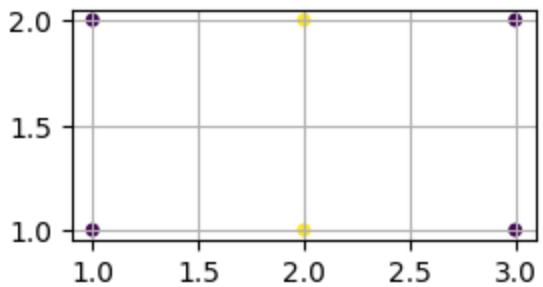
TA11 Actividades de árboles, bosques, bagging y boosting

2 Árboles

Cuestión. Sea un problema de clasificación de datos 2d en $C = 2$ clases, $y \in \{1, 2\}$, para el que se está construyendo un árbol de clasificación. El algoritmo de aprendizaje se halla procesando un nodo i cuyo conjunto de datos, $\mathcal{D}_i$, consta de 4 datos de la clase 1, $\{(1,1)^t, (1,2)^t, (3,1)^t, (3,2)^t\}$, y 2 de la clase 2, $\{(2,1)^t, (2,2)^t\}$:

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
X = np.array([[1, 1], [1, 2], [3, 1], [3, 2], [2, 1], [2, 2]]); y =  np.array([1, 1, 1, 1, 2, 2])
plt.figure(figsize=(3, 1.5)); plt.grid(); plt.scatter(*X.T, c=y, s=16);
```



Sea H_i la impureza del nodo i medida como la entropía de distribución empírica de las clases. Sea (j_i, t_i) un split óptimo en términos de reducción de impureza. Indica la afirmación correcta:

1. $H_i > 1$ y $(j_i, t_i) = (1, 1.5)$
2. $H_i > 1$ y $(j_i, t_i) = (2, 1.5)$
3. $H_i \leq 1$ y $(j_i, t_i) = (1, 1.5)$
4. $H_i \leq 1$ y $(j_i, t_i) = (2, 1.5)$

Solución: La 3.

$$H_i = -\hat{\pi}_{i1} \log_2 \hat{\pi}_{i1} - \hat{\pi}_{i2} \log_2 \hat{\pi}_{i2} = -\frac{4}{6} \log_2 \frac{4}{6} - \frac{2}{6} \log_2 \frac{2}{6} = 0.9183$$

- $j_i = 1, t_i = 1.5$

$$\begin{aligned} H(\mathcal{D}_i^L(1, 1.5)) &= -1 \log_2 1 - 0 \log_2 0 = 0 \\ H(\mathcal{D}_i^R(1, 1.5)) &= -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1 \\ \Delta(1, 1.5) &= 0.9183 - \frac{2}{6} \cdot 0 - \frac{4}{6} \cdot 1 = 0.2516 \end{aligned}$$

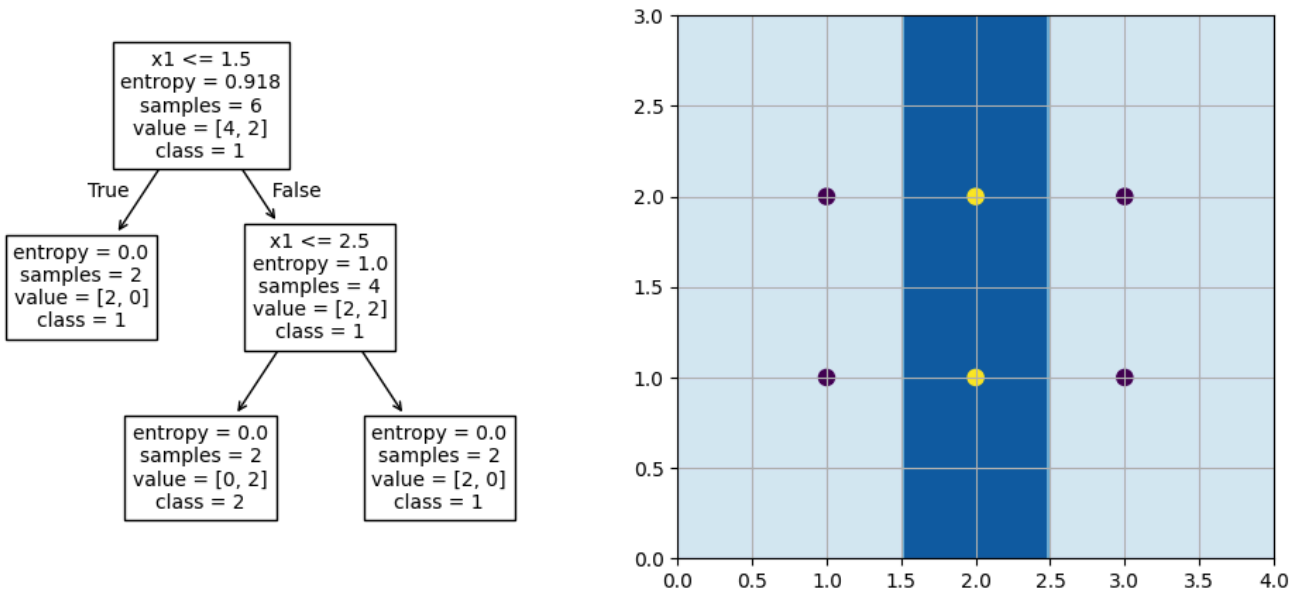
- $j_i = 1, t_i = 2.5 \qquad \Delta(1, 2.5) = 0.2516$
- $j_i = 2, t_i = 1.5$

$$\begin{aligned} H(\mathcal{D}_i^L(2, 1.5)) &= -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} = 0.9183 \\ H(\mathcal{D}_i^R(2, 1.5)) &= 0.9183 \\ \Delta(2, 1.5) &= 0.9183 - \frac{3}{6} \cdot 0.9183 - \frac{3}{6} \cdot 0.9183 = 0 \end{aligned}$$

- Óptimo: cualquiera de los dos primeros

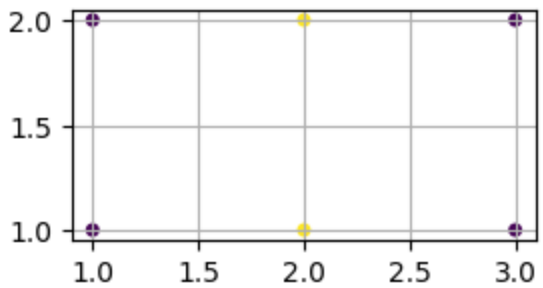
In [ ]:

```
import numpy as np; import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
X = np.array([[1, 1], [1, 2], [3, 1], [3, 2], [2, 1], [2, 2]]); y =  np.array([1, 1, 1, 1, 2, 2])
dt = DecisionTreeClassifier(criterion='entropy', max_depth=2, random_state=23).fit(X, y)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
plot_tree(dt, feature_names=list(('x1', 'x2')), class_names=list(('1', '2')), ax=axes[0], fontsize=10);
xx, yy = np.meshgrid(np.linspace(0, 4, num=100), np.linspace(0, 3, num=100))
Z = dt.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape); axes[1].grid()
cp = axes[1].contourf(xx, yy, Z, 2, cmap='Blues'); axes[1].scatter(*X.T, c=y, s=64);
```



Cuestión. Sea un problema de clasificación de datos 2d en $C = 2$ clases, $y \in \{1, 2\}$, para el que se está construyendo un árbol de clasificación. El algoritmo de aprendizaje se halla procesando un nodo i cuyo conjunto de datos, $\mathcal{D}_i$, consta de 4 datos de la clase 1, $\{(1,1)^t, (1,2)^t, (3,1)^t, (3,2)^t\}$, y 2 de la clase 2, $\{(2,1)^t, (2,2)^t\}$:

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
X = np.array([[1, 1], [1, 2], [3, 1], [3, 2], [2, 1], [2, 2]]); y =  np.array([1, 1, 1, 1, 2, 2])
plt.figure(figsize=(3, 1.5)); plt.grid(); plt.scatter(*X.T, c=y, s=16);
```



Sea G_i la impureza del nodo i medida mediante el índice Gini (error de clasificación esperado). Sea (j_i, t_i) un split óptimo en términos de reducción de impureza. Indica la afirmación correcta:

1. $G_i > 0.4$ y $(j_i, t_i) = (1, 1.5)$
2. $G_i > 0.4$ y $(j_i, t_i) = (2, 1.5)$
3. $G_i \leq 0.4$ y $(j_i, t_i) = (1, 1.5)$
4. $G_i \leq 0.4$ y $(j_i, t_i) = (2, 1.5)$

Solución: La 1.

$$G_i = 1 - \hat{\pi}_{i1}^2 - \hat{\pi}_{i2}^2 = 1 - (4/6)^2 - (2/6)^2 = 1 - 20/36 = 16/36 = 4/9 = 0.44$$

En el eje horizontal (dimensión $d = 1$) tenemos dos posibles splits que dejan dos datos de la clase 1 a un lado y el resto al otro lado, por lo que producen la misma reducción de impureza. Tomemos, por ejemplo, el split basado en el umbral $t = 1.5$. En este caso, dos datos de la clase 1 van al hijo izquierdo y el resto al derecho. Las impurezas de los hijos y reducción de impureza respecto al padre son:

$$\begin{aligned}
G(\mathcal{D}_i^L(1, 1.5)) &= 1 - (2/2)^2 - (0/2)^2 = 0 \\
G(\mathcal{D}_i^R(1, 1.5)) &= 1 - (2/4)^2 - (2/4)^2 = 1/2 \\
\Delta &= G_i - 2/6\, G(\mathcal{D}_i^L(1, 1.5)) - 4/6\, G(\mathcal{D}_i^R(1, 1.5)) \\
&= 4/9 - 2/6 \cdot 0 - 4/6 \cdot 1/2 = 1/9 = 0.11
\end{aligned}$$

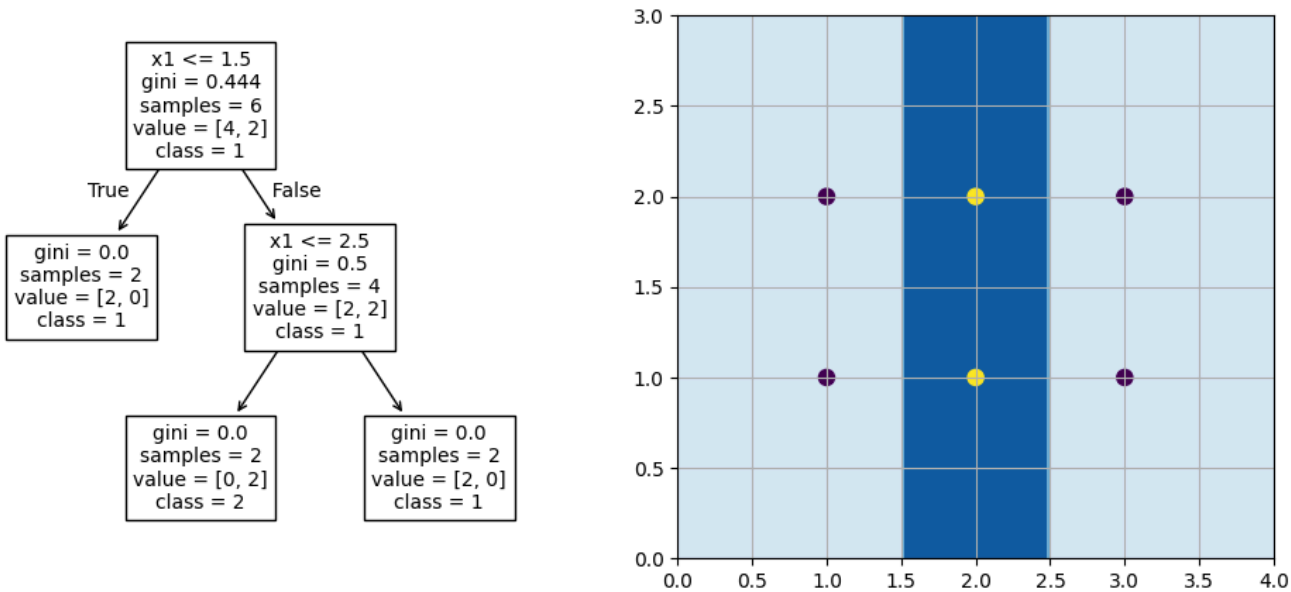
En el eje vertical ($d = 2$) tenemos un único split que deja ambos hijos con dos datos de la clase 1 y uno de la 2:

$$\begin{aligned}
G(\mathcal{D}_i^L(1, 1.5)) &= 1 - (2/3)^2 - (1/3)^2 = 1 - 5/9 = 4/9 \\
G(\mathcal{D}_i^R(1, 1.5)) &= G(\mathcal{D}_i^L(1, 1.5)) \\
\Delta &= G_i - 3/6\, G(\mathcal{D}_i^L(1, 1.5)) - 3/6\, G(\mathcal{D}_i^R(1, 1.5)) \\
&= 4/9 - 3/6 \cdot 4/9 - 3/6 \cdot 4/9 = 0
\end{aligned}$$

Por tanto, un split óptimo consiste en particionar los datos con la primera variable (horizontal) y umbral $t = 1.5$; también sería óptimo emplear el umbral $t = 2.5$.

In [ ]:

```
import numpy as np; import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
X = np.array([[1, 1], [1, 2], [3, 1], [3, 2], [2, 1], [2, 2]]); y =  np.array([1, 1, 1, 1, 2, 2])
dt = DecisionTreeClassifier(criterion='gini', max_depth=2, random_state=23).fit(X, y)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
plot_tree(dt, feature_names=list(('x1', 'x2')), class_names=list(('1', '2')), ax=axes[0], fontsize=10);
xx, yy = np.meshgrid(np.linspace(0, 4, num=100), np.linspace(0, 3, num=100))
Z = dt.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape); axes[1].grid()
cp = axes[1].contourf(xx, yy, Z, 2, cmap='Blues'); axes[1].scatter(*X.T, c=y, s=64);
```



Cuestión. Sea un problema de clasificación de datos 3d en $C = 2$ clases, $y \in \{1, 2\}$, para el que se está construyendo un árbol de clasificación. El algoritmo de aprendizaje se halla procesando un nodo i cuyo conjunto de datos, $\mathcal{D}_i$, consta de 2 datos de la clase 1, $\{(1,1,2)^t, (1,2,1)^t\}$, y 2 de la clase 2, $\{(2,1,2)^t, (2,2,1)^t\}$. Sea G_i la impureza del nodo i medida mediante el índice Gini (error de clasificación esperado). Sea (j_i, t_i) un split óptimo en términos de reducción de impureza. Indica la afirmación correcta:

1. $G_i > 0.5$ y $(j_i, t_i) = (1, 1.5)$
2. $G_i > 0.5$ y $(j_i, t_i) = (2, 1.5)$
3. $G_i \leq 0.5$ y $(j_i, t_i) = (1, 1.5)$
4. $G_i \leq 0.5$ y $(j_i, t_i) = (2, 1.5)$

Solución: La 3. El nodo i tiene dos datos de la clase 1 y otros de la 2, por lo que resulta máximamente impuro:

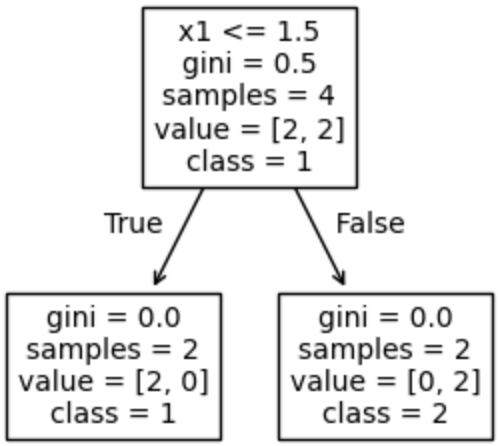
$$G_i = 1 - \hat{\pi}_{i1}^2 - \hat{\pi}_{i2}^2 = 1 - (2/4)^2 - (2/4)^2 = 1 - 1/2 = 1/2$$

Es fácil determinar los posibles splits del nodo i si tenemos en cuenta que, en las tres dimensiones, los datos están en 1 y 2 únicamente. Por tanto, solo cabe un split por cada dimensión, en 1.5 por ejemplo. El split en la dimensión 1 deja los dos datos de la clase 1 a un lado y los dos datos de la clase 2 al otro; esto es, produce dos nodos hijos puros. Al contratrio, los splits en las dimensiones 2 y 3 producen hijos con un dato de cada clase; esto es, máximamente impuros, como el nodo i. En definitiva, el split de la dimensión 1 es óptimo y produce la máxima reducción de impureza posible:

$$G(\mathcal{D}_i^L(1, 1.5)) = 1 - (2/2)^2 - (0/2)^2 = 0 \qquad G(\mathcal{D}_i^R(1, 1.5)) = 1 - (0/2)^2 - (2/2)^2 = 0$$

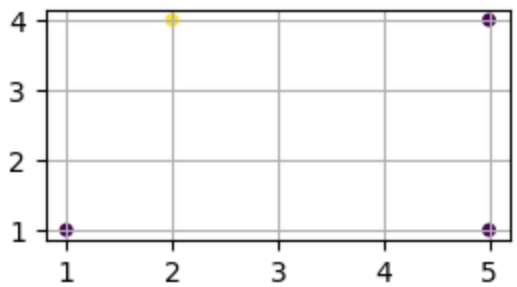
$$\Delta = G_i - 2/4\, G(\mathcal{D}_i^L(1, 1.5)) - 2/4\, G(\mathcal{D}_i^R(1, 1.5)) = 1/2 - 0 - 0 = 1/2$$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from sklearn.tree import DecisionTreeClassifier, plot_tree
X = np.array([[1,1,2],[1,2,1],[2,1,2],[2,2,1]]); y =  np.array([1,1,2,2]); _, ax = plt.subplots(figsize=(3.5, 3
dt = DecisionTreeClassifier(criterion='gini', max_depth=2, random_state=23).fit(X, y)
plot_tree(dt, feature_names=list(('x1', 'x2')), class_names=list(('1', '2')), fontsize=10, ax=ax);
```



Cuestión. Sea un problema de regresión de datos 2d en $\mathbb{R}$, para el que se está construyendo un árbol de regresión. El algoritmo de aprendizaje se halla procesando un nodo i de conjunto de datos $\mathcal{D}_i = \{((1,1)^t, 1),\ ((2,4)^t, 2),\ ((5,1)^t, 1),\ ((5,4)^t, 1)\}$:

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
X = np.array([[1, 1], [2, 4], [5, 1], [5, 4]]); y =  np.array([1, 2, 1, 1])
plt.figure(figsize=(3, 1.5)); plt.grid(); plt.scatter(*X.T, c=y, s=16);
```



Sea $c(\mathcal{D}_i)$ el coste del nodo i medido mediante el error cuadrático medio (con respecto a la respuesta media del nodo). Sea (j_i, t_i) un split óptimo en términos de reducción de coste. Indica la afirmación correcta:

1. $c(\mathcal{D}_i) > 0.1$ y $(j_i, t_i) = (1, 1.5)$
2. $c(\mathcal{D}_i) > 0.1$ y $(j_i, t_i) = (1, 3.5)$
3. $c(\mathcal{D}_i) \leq 0.1$ y $(j_i, t_i) = (1, 1.5)$
4. $c(\mathcal{D}_i) \leq 0.1$ y $(j_i, t_i) = (1, 3.5)$

Solución: La 2.

$$\hat{y} = \frac{5}{4} = 1.25 \qquad c(\mathcal{D}_i) = \frac{1}{4}(3(1-1.25)^2 + (2-1.25)^2) = 3/16 = 0.1875$$

1. $j_i = 1, t_i = 3.5$

$$\begin{aligned} c(\mathcal{D}_i^L(1, 3.5)) &= \frac{1}{2}((1-1.5)^2 + (2-1.5)^2) = 1/4 \\ c(\mathcal{D}_i^R(1, 3.5)) &= \frac{1}{2}(2(1-1)^2) = 0 \\ \Delta(1, 3.5) &= 3/16 - 2/4 \cdot 1/4 - 2/4 \cdot 0 = 1/16 = 0.0625 \end{aligned}$$

2. $j_i = 1, t_i = 1.5$

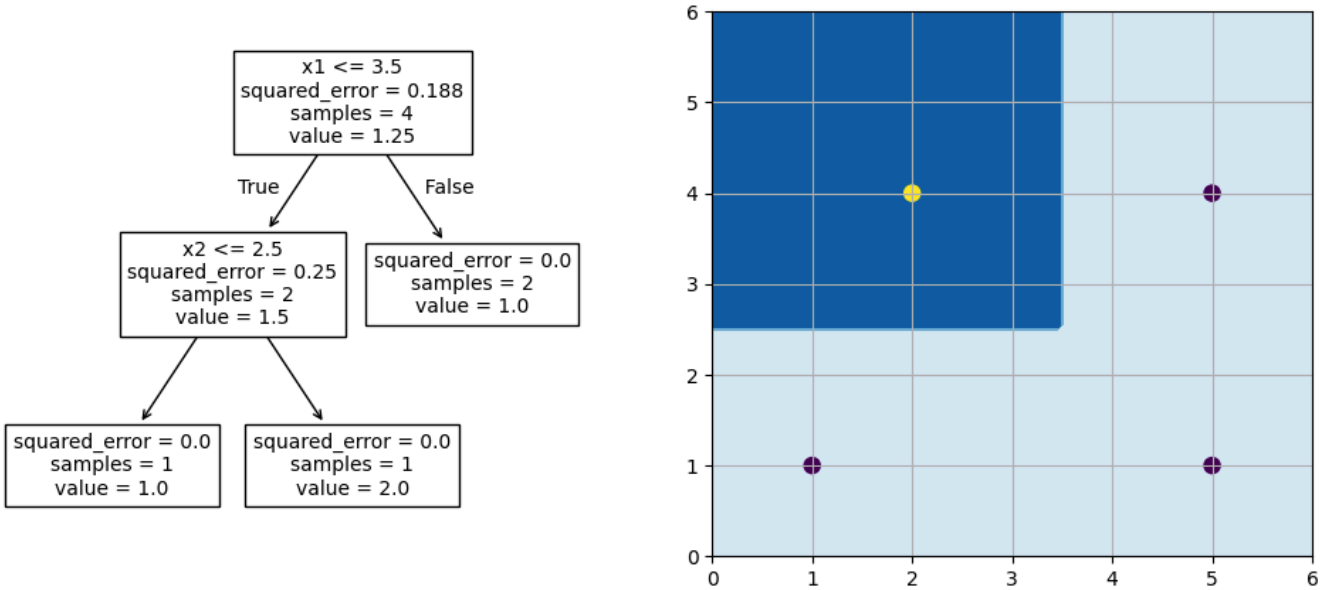
$$\begin{aligned} c(\mathcal{D}_i^L(1, 1.5)) &= \frac{1}{1}(1-1)^2 = 0 \\ c(\mathcal{D}_i^R(1, 1.5)) &= \frac{1}{3}((2-4/3)^2 + 2(1-4/3)^2) = 2/9 \\ \Delta(1, 1.5) &= 3/16 - 1/4 \cdot 0 - 3/4 \cdot 2/9 = 0.0208 \end{aligned}$$

3. $j_i = 2, t_i = 2.5 \rightarrow \Delta = 1/16$

Split óptimo: el 1 o el 3

In [ ]:

```
import numpy as np; import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor, plot_tree
X = np.array([[1, 1], [2, 4], [5, 1], [5, 4]]); y =  np.array([1, 2, 1, 1])
dt = DecisionTreeRegressor(criterion='squared_error', max_depth=2, random_state=23).fit(X, y)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
plot_tree(dt, feature_names=list(('x1', 'x2')), ax=axes[0], fontsize=10);
xx, yy = np.meshgrid(np.linspace(0, 6, num=100), np.linspace(0, 6, num=100))
Z = dt.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape); axes[1].grid()
cp = axes[1].contourf(xx, yy, Z, 2, cmap='Blues'); axes[1].scatter(*X.T, c=y, s=64);
```



5 Boosting

Cuestión. Sea un problema de regresión de datos 1d para el que tenemos $N = 5$ datos de entrenamiento:

n	1	2	3	4
x_n	1	2	3	4
y	0	4	3	5

Se ha aplicado boosting mínimos cuadrados para ajustar un modelo adaptativo $f(x)$ con $M = 2$ modelos base de tipo *stump* (tocón), esto es, árboles de regresión de profundidad uno. Los modelos base ajustados son:

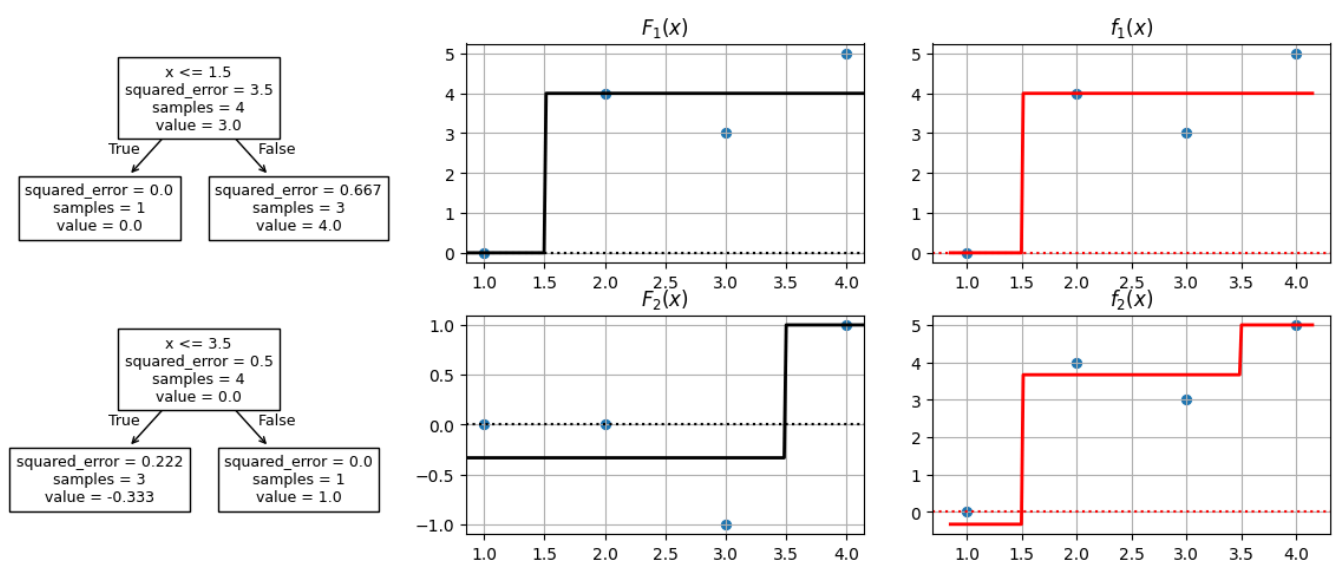
$$F_1(x) = \begin{cases} 0 & \text{si } x \leq 1.5 \\ 4 & \text{si no} \end{cases} \qquad F_2(x) = \begin{cases} -1/3 & \text{si } x \leq 3.5 \\ 1 & \text{si no} \end{cases}$$

Dado $x = 2.5$, la predicción del modelo ajustado es:

1. $f(x) < 2$
2. $2 \leq f(x) < 3$
3. $3 \leq f(x) < 4$
4. $f(x) \geq 4$

Solución: La 3; $f(2.5) = F_1(2.5) + F_2(2.5) = 4 - 1/3 = 11/3 = 3.67$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from sklearn.tree import DecisionTreeRegressor, plot_tree
Xy = np.array([[1,0],[2,4],[3,3],[4,5]], dtype=float); X = Xy[:, 0, np.newaxis]; y = Xy[:, 1]
M = 2; fig, axs = plt.subplots(M, 3, figsize=(12, 4.75)); fig.tight_layout()
res = np.copy(y); tt = []; ax = axs[0, 0]; ax.scatter(X, y)
x_min, x_max = ax.get_xlim(); xx = np.linspace(x_min, x_max, 200)
for m in range(M):
    t = DecisionTreeRegressor(max_depth=1, random_state=42).fit(X, res); tt.append(t)
    plot_tree(t, feature_names=list(('x')), ax=axs[m, 0], fontsize=9)
    ax = axs[m, 1]; ax.grid(); ax.axhline(y=0, color='k', linestyle=':')
    ax.set_title('$F_{}(x)$'.format(m+1)); ax.set_xlim(x_min, x_max); ax.scatter(X, res, s=32)
    res_pred = t.predict(xx.reshape(-1, 1)); ax.plot(xx, res_pred, 'k-', linewidth=2)
    ax = axs[m, 2]; ax.grid(); ax.axhline(y=0, color='r', linestyle=':')
    ax.set_title('$f_{}(x)$'.format(m+1)); ax.scatter(X, y, s=32)
    y_pred = sum(t.predict(xx.reshape(-1, 1)) for t in tt)
    ax.plot(xx, y_pred, 'r-', linewidth=2); res -= t.predict(X)
```



Cuestión. Sea un problema de regresión de datos 2d para el que tenemos $N = 5$ datos de entrenamiento:

n	1	2	3	4	5
$\boldsymbol{x}_n^t$	$(2,3)$	$(4,4)$	$(6,6)$	$(8,2)$	$(8,7)$
$\tilde{y}$	1	-1	1	-2	-1

Se ha aplicado boosting mínimos cuadrados para ajustar un modelo adaptativo $f(\boldsymbol{x})$ con $M = 3$ modelos base de tipo *stump* (tocón), esto es, árboles de regresión de profundidad uno. Los modelos base ajustados son:

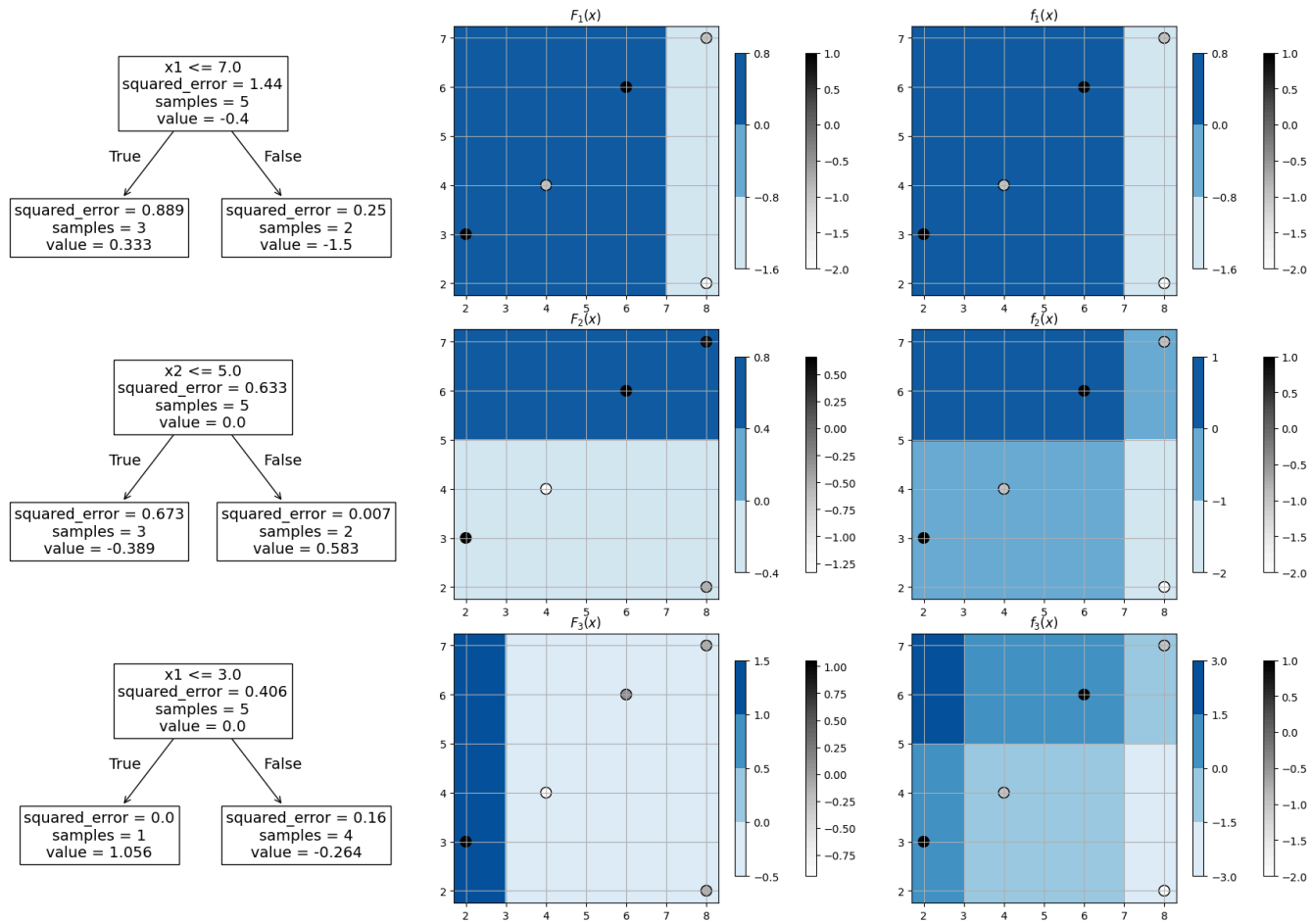
$$F_1(\boldsymbol{x}) = \begin{cases} 1/3 & \text{si } x_1 \leq 7 \\ -1.5 & \text{si no} \end{cases} \qquad F_2(\boldsymbol{x}) = \begin{cases} -0.389 & \text{si } x_2 \leq 5 \\ 0.583 & \text{si no} \end{cases} \qquad F_3(\boldsymbol{x}) = \begin{cases} 1.056 & \text{si } x_1 \leq 3 \\ -0.264 & \text{si no} \end{cases}$$

Dado $\boldsymbol{x} = (4,6)^t$, la predicción del modelo ajustado es:

1. $f(\boldsymbol{x}) < -1$
2. $-1 \leq f(\boldsymbol{x}) < 0$
3. $0 \leq f(\boldsymbol{x}) < 1$
4. $f(\boldsymbol{x}) \geq 1$

Solución: La 2; $f(4,6) = F_1(4,6) + F_2(4,6) + F_3(4,6) = 1/3 + 0.583 - 0.264 = 0.6523$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor, plot_tree
Xy = np.array([[2,3,1],[4,4,-1],[6,6,1],[8,7,-1],[8,2,-2]], dtype=float)
X = Xy[:, :2]; y = Xy[:, 2]; M = 3
fig, axs = plt.subplots(M, 3, figsize=(6*M, 12)); fig.tight_layout()
res = np.copy(y); tt = []; ax = axs[0, 0]; ax.scatter(*X.T, c=y, s=64)
x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
X_test = np.c_[xx.ravel(), yy.ravel()]
for m in range(M):
    t = DecisionTreeRegressor(max_depth=1, random_state=23).fit(X, res); tt.append(t)
    plot_tree(t, feature_names=list(('x1', 'x2')), ax=axs[m, 0], fontsize=14)
    ax = axs[m, 1]; ax.grid(); ax.set_title('$F_{}(x)$'.format(m+1))
    F = t.predict(X_test).reshape(xx.shape)
    cp = ax.contourf(xx, yy, F, 2, cmap='Blues')
    sp = ax.scatter(*X.T, c=res.reshape(1, -1), cmap='Greys', edgecolors='black', s=100)
    plt.colorbar(sp, ax=ax, shrink=0.8); plt.colorbar(cp, ax=ax, shrink=0.8)
    ax = axs[m, 2]; ax.grid(); ax.set_title('$f_{}(x)$'.format(m+1))
    f = sum(t.predict(X_test) for t in tt).reshape(xx.shape)
    cp = ax.contourf(xx, yy, f, 2, cmap='Blues')
    sp = ax.scatter(*X.T, c=y, cmap='Greys', edgecolors='black', s=100)
    plt.colorbar(sp, ax=ax, shrink=0.8); plt.colorbar(cp, ax=ax, shrink=0.8)
    res -= t.predict(X)
```



Cuestión. Sea un problema de clasificación de datos 2d en $C = 2$ clases, $y \in \{-1, +1\}$, para el que tenemos $N = 5$ datos, $\mathcal{D} = \{((2,3)^t, 1), ((4,4)^t, -1), ((6,6)^t, 1), ((8,2)^t, -1), ((8,7)^t, -1)\}$. Se ha aplicado Adaboost para ajustar un modelo adaptativo $f(\boldsymbol{x})$ con $M = 3$ modelos base de tipo stump:

$$F_1(\boldsymbol{x}) = 2\mathbb{I}(x_1 \leq 7) - 1 \qquad F_2(\boldsymbol{x}) = 2\mathbb{I}(x_1 \leq 3) - 1 \qquad F_3(\boldsymbol{x}) = 2\mathbb{I}(x_2 \leq 5) - 1$$

Para $m \in \{1, 2, 3\}$, los pesos de los datos, w_{nm} $(n = 1, \ldots, 5)$, la clasificación de los mismos según F_m, el error ponderado err_m y el peso de F_m en el ensamble f, β_m, son:

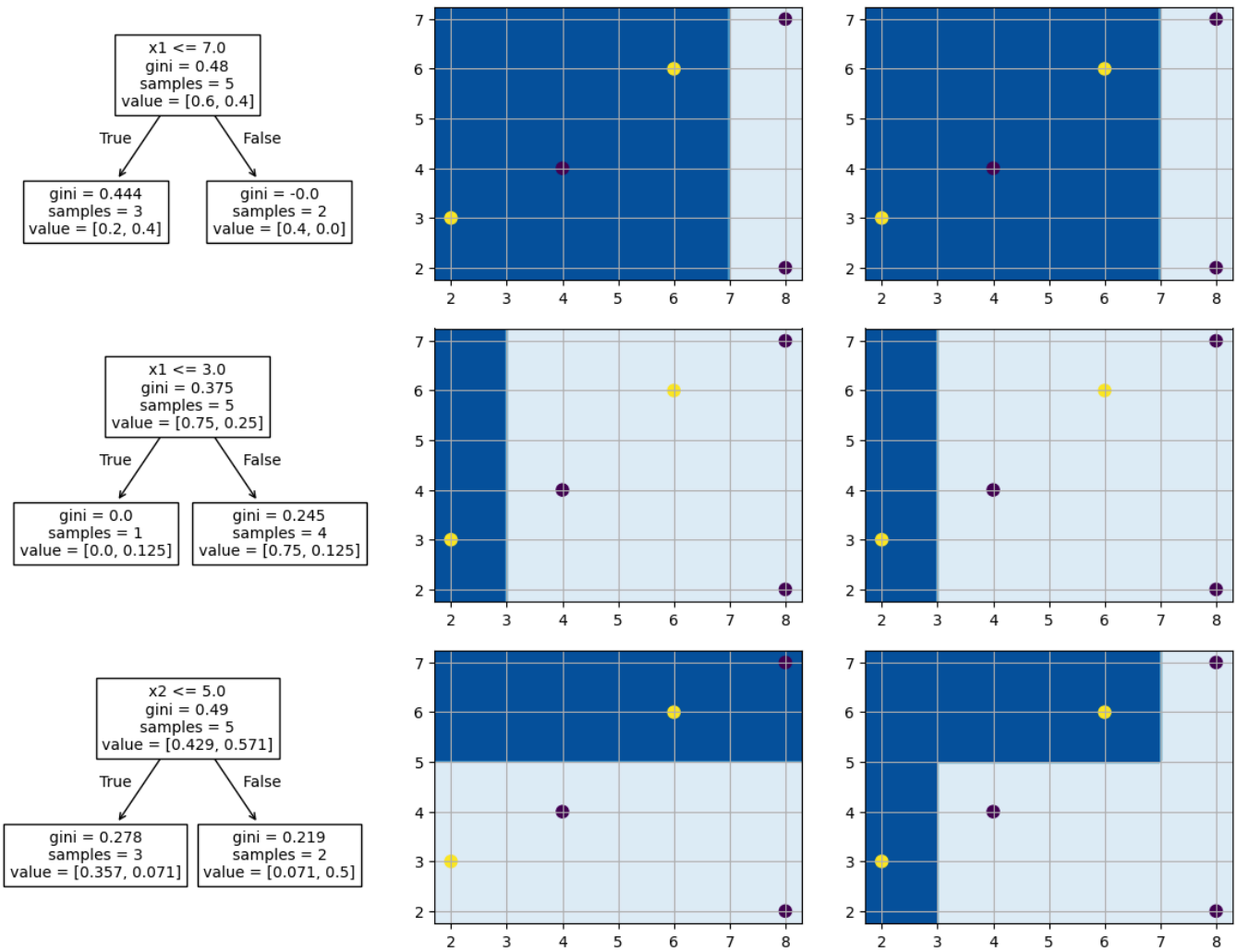
n	1	2	3	4	5	
$\boldsymbol{x}_n^t$	$(2,3)$	$(4,4)$	$(6,6)$	$(8,2)$	$(8,7)$	
$\tilde{y}$	1	-1	1	-1	-1	
w_{n1}	0.2	0.2	0.2	0.2	0.2	
$F_1(\boldsymbol{x}_n)$	1	1	1	-1	-1	$\mathrm{err}_1 = 0.2 \quad \beta_1 = \frac{1}{2}\log\frac{0.8}{0.2} = 0.6931$
w_{n2}	0.2	0.8	0.2	0.2	0.2	$w_{22} = w_{21}\exp(2\beta_1) = 0.8$
$F_2(\boldsymbol{x}_n)$	1	-1	-1	-1	-1	$\mathrm{err}_2 = \frac{1}{1.6}0.2 = 0.125 \quad \beta_2 = \frac{1}{2}\log\frac{0.875}{0.125} = 0.9730$
w_{n3}	0.2	0.8	1.4	0.2	0.2	$w_{33} = w_{32}\exp(2\beta_2) = 1.4$
$F_3(\boldsymbol{x}_n)$	1	1	-1	1	-1	$\mathrm{err}_3 = \frac{2.4}{2.8} = 0.8571 \quad \beta_3 = \frac{1}{2}\log\frac{1-0.8571}{0.8571} = -0.8959$

Considera el modelo ajustado tanto con los dos primeros modelos base, f_2, como con el ensamble completo, f_3. Las predicciones de ambos para $\boldsymbol{x} = (4,6)^t$ son:

1. $f_2(\boldsymbol{x}) = -1$ y $f_3(\boldsymbol{x}) = -1$
2. $f_2(\boldsymbol{x}) = -1$ y $f_3(\boldsymbol{x}) = 1$
3. $f_2(\boldsymbol{x}) = 1$ y $f_3(\boldsymbol{x}) = -1$
4. $f_2(\boldsymbol{x}) = 1$ y $f_3(\boldsymbol{x}) = 1$

Solución: La 2; $f_3(\boldsymbol{x}) = \operatorname{sgn}(0.6931 \cdot 1 + 0.9730 \cdot (-1) + (-0.8959) \cdot (-1)) = 1,\ f_2(\boldsymbol{x}) = -1$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
from sklearn.tree import plot_tree; from sklearn.ensemble import AdaBoostClassifier
Xy = np.array([[2,3,1],[4,4,-1],[6,6,1],[8,7,-1],[8,2,-1]]); X = Xy[:, :2]; y = Xy[:, 2]
M = 3; fig, axs = plt.subplots(M, 3, figsize=(4*M, 9)); fig.tight_layout()
f = AdaBoostClassifier(n_estimators=M, random_state=23).fit(X, y)
ax = axs[0, 0]; ax.scatter(*X.T, c=y, s=64); x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
y_test = np.c_[xx.ravel(), yy.ravel()]; y_pred = f.staged_predict(y_test)
for m, y_pred in enumerate(y_pred):
    plot_tree(f.estimators_[m], feature_names=list(('x1', 'x2')), ax=axs[m, 0], fontsize=10)
    ax = axs[m, 1]; ax.grid(); Z = f.estimators_[m].predict(y_test).reshape(xx.shape)
    cp = ax.contourf(xx, yy, Z, 2, cmap='Blues'); ax.scatter(*X.T, c=y, s=64)
    ax = axs[m, 2]; ax.grid(); Z = y_pred.reshape(xx.shape)
    cp = ax.contourf(xx, yy, Z, 2, cmap='Blues'); ax.scatter(*X.T, c=y, s=64)
```



Cuestión. Los árboles constituyen un estimador de alta varianza ya que pequeñas perturbaciones de los datos de entrenamiento resultan en predicciones muy distintas. El aprendizaje de ensambles reduce la varianza de los árboles promediando $|\mathcal{M}|$ modelos base $\{f_m\}$, $f(y \mid \boldsymbol{x}) = \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} f_m(y \mid \boldsymbol{x})$. Indica la respuesta incorrecta (o escoge la última opción si las tres primeras son correctas).

1. En regresión, el promediado suele presentar un sesgo similar al de los modelos base, pero mejor precisión por la menor varianza.
2. En clasificación se usa el voto mayoritario o método comité, que incrementa la precisión de los modelos base, especialmente cuando sus errores de predicción no se hallan correlados.
3. Bagging, bosques aleatorios y boosting aprenden ensambles de árboles con el fin de reducir la varianza. Además, bosques aleatorios y boosting adaptan el ajuste de cada modelo base teniendo en cuenta los modelos base ajustados previamente. De esta manera, no solo consiguen reducir la varianza, sino que también reducen el sesgo del ensamble resultante.
4. Todas son correctas.

Solución: La 3; bosques aleatorios no reduce el sesgo; el resto de afirmaciones está bien.